

3Dアクションゲーム 企画書

プログラム設計・技術仕様および実装内容に関する解説資料

本資料の目的

本資料は、作品の概要に加え、開発において採用したプログラムアーキテクチャ、オブジェクト指向設計、数学的处理、コード品質の管理手法について具体的に解説した技術仕様資料です。

1. 作品概要と開発環境

ジャンル	3Dアクションゲーム	動作環境	Windows 10 / 11
開発環境	C++ / Visual Studio 2022	使用ライブラリ	DxLib / Effekseer

フィールド上のターゲットを制限時間内に捕獲しスコアを獲得する3Dアクションゲームです。一定スコア到達によるフィーバータイムや各種ギミックの発生など、ゲーム展開を変化させる構造を取り入れています。ライブラリの基本処理を活用しつつ、ゲームループや各種制御系を自作クラスとして構築しています。

2. 全体アーキテクチャ

ステートパターンによるシーン遷移管理

SceneManagerを中心に、TitleScene, TutorialScene, GameScene, ResultSceneなどの各シーンを状態として分離して管理しています。各画面の初期化、更新、描画、終了処理をカプセル化し、安全でスムーズな画面切り替えを実現しました。

サービスロケーターパターンの採用

ServiceLocatorを導入し、InputManagerやSoundManagerなどのシステムへアクセスする設計を採用しました。グローバル変数やシングルトンパターンによる密結合を避け、各コンポーネント間の独立性を維持しています。

3. オブジェクト指向に基づくクラス設計

すべての3Dオブジェクトの基底クラスとしてObject3Dを定義しています。継承を用いてPlayer3DやCowなどのクラスを派生させ、ポリモーフィズムを活用してObjectManagerで一括して更新および描画を行う仕組みを構築しました。

```
// オブジェクト基底クラスの設計例
class Object3D { public: virtual ~Object3D() = default; virtual void Initialize() = 0; virtual void Update(float delta_time) = 0; virtual void Draw() const = 0; const Vector3& GetPosition() const { return position_; } bool IsActive() const { return is_active_; } protected: Vector3 position_{0.0f, 0.0f, 0.0f}; Vector3 rotation_{0.0f, 0.0f, 0.0f}; bool is_active_{true}; };
```

- **Player3D:** プレイヤーの入力処理、移動計算、旋回処理を担当します。
- **Cow:** フィールド上のターゲットとして、配置や状態の管理を行います。

4. 当たり判定のモジュール化

CapsuleColliderやSphereColliderなどの判定要素をコンポーネント化し、オブジェクトに柔軟に組み込める設計にしています。ColliderManagerを介して効率的に衝突判定の計算を行っています。

5. 3D数学・カメラ制御

ベクトル演算を用いたプレイヤーの移動計算および、プレイヤーを追従する3Dカメラの視点計算を実装しました。また、竜巻や雷などのギミックにおいて動的に座標計算を行うアルゴリズムを取り入れています。

6. 大量のオブジェクト管理

フィールド上に多数のターゲットやエフェクトが存在する状態でも処理落ちを防ぐため、オブジェクトの管理基盤を整えています。

一元管理マネージャーの導入

CowManagerやAnimalManagerなどを設け、ターゲットの生成座標やプレイヤーへの追尾などのAI挙動を一元管理する設計としています。

7. コードの品質・可読性へのこだわり

C++の業界標準であるGoogle C++ Style Guideに準拠したルールを設定し、チーム開発や長期的な保守を意識したコードベースを構築しました。

規約項目	ルール・方針	具体例
メンバ変数	末尾にアンダーバーを付与	is_out_of_bounds_
定数	プレフィックスkを付与	kJumpPower
メモリ管理	マネージャークラスでの確実に解放処理	デストラクタでの自動破棄

```
// メモリ管理と資源解放の例
class ModelResource { public: explicit ModelResource(const std::string& file_path) { model_handle_ = MV1LoadModel(file_path.c_str()); } ~ModelResource() { if (model_handle_ != -1) { MV1DeleteModel(model_handle_); } } ModelResource(const ModelResource&) = delete; ModelResource& operator=(const ModelResource&) = delete; private: int model_handle_{-1}; };
```

6. 大量のオブジェクト管理

フィールド上に多数のターゲットやエフェクトが存在する状態でも処理落ちを防ぐため、オブジェクトの管理基盤を整えています。

一元管理マネージャーの導入

CowManagerやAnimalManagerなどを設け、ターゲットの生成座標やプレイヤーへの追尾などのAI挙動を一元管理する設計としています。

7. コードの品質・可読性へのこだわり

C++の業界標準であるGoogle C++ Style Guideに準拠したルールを設定し、チーム開発や長期的な保守を意識したコードベースを構築しました。

規約項目	ルール・方針	具体例
メンバ変数	末尾にアンダーバーを付与	is_out_of_bounds_
定数	プレフィックスkを付与	kJumpPower
メモリ管理	マネージャークラスでの確実に解放処理	デストラクタでの自動破棄

```
// メモリ管理と資源解放の例
class ModelResource { public: explicit ModelResource(const std::string& file_path) { model_handle_ = MV1LoadModel(file_path.c_str()); } ~ModelResource() { if (model_handle_ != -1) { MV1DeleteModel(model_handle_); } } ModelResource(const ModelResource&) = delete; ModelResource& operator=(const ModelResource&) = delete; private: int model_handle_{-1}; };
```

8. 就職活動における総括

使用技術	C++17 / DxLib / Effekseer for DxLib
開発ツール・ビルド結果	Visual Studio 2022 0エラー 0警告 Release x64

開発成果および取り組みの総括

リーダーとして開発基盤を整備し、チーム内の円滑な分業と品質管理を主導しました。また自作エンジン同等の技術力を養うため、当たり判定やAI状態遷移をゼロから実装し、警告ゼロの高精度なコードベースを維持しています。